Střední průmyslová škola a Gymnázium Na Třebešíně

MATURITNÍ PRÁCE S OBHAJOBOU



INTERAKTIVNÍ DIGITÁLNÍ POZNÁMKOVÝ BLOK S INTEGROVANÝM AI ASISTENTEM PRO VÝUKU

Vypracoval:	Kostiantyn Kudinov
Vedoucí:	Mgr. Jan Souhrada
Třída:	4. G
Školní rok:	2026 / 2027

1 Cíle MPsO

Cílem projektu je navrhnout, implementovat a otestovat moderní multiplatformní aplikaci pro digitální psaní studijních poznámek a interaktivní tabuli (whiteboard) s integrovaným AI asistentem pro výuku. Aplikace spojuje vysoce výkonné vektorové kreslení bez znatelné vstupní latence, správu vícestránkových sešitů a lokálně orientovanou synchronizaci v reálném čase.

V souladu se schváleným zadáním práce byly stanoveny následující konkrétní cíle:

• **Cíl 1 (Vektorové vykreslovací jádro a multiplatformní klient):** Vytvořit klientskou aplikaci pro web i desktop (Linux, Windows, macOS) se stabilní frekvencí 60 FPS při práci s tisíci tahů a hardwarovou kompenzací rozlišení (DPR super-sampling) pro HiDPI displeje.
• **Cíl 2 (Hybridní prostorový model sešitu):** Implementovat přepínání mezi nekonečnou plochou (ZUI) a stránkovaným sešitem formátu ISO 216 A4, včetně importu a ručního anotování PDF a exportu do formátů .wnb a PDF.
• **Cíl 3 (Local-First architektura a síťová synchronizace):** Zajistit plnohodnotný offline provoz s lokálním ukládáním (IndexedDB / SQLite WAL) a kolaborativní synchronizaci poznámek mezi zařízeními v reálném čase přes Yjs CRDT a WebSockets.
• **Cíl 4 (Multimodální AI asistent pro výuku):** Propojit poznámkový blok s Vision-Language modelem pro kontextové vysvětlování ručně psaných výpočtů, schémat a poznámek označených přímo na plátně formou sokratovského tutora.
• **Cíl 5 (Generátor procvičovacích úloh):** Vytvořit modul aktivního vybavování (active recall) pro automatické generování interaktivních procvičovacích otázek a kartiček (flashcards) z obsahu sešitu.

2 Postupy a metodiky pro dosažení cílů MPsO

Architektonický návrh a volba technologií: Základem aplikace je moderní hybridní architektura oddělující systémový backend od klientského rozhraní. Pro desktopový backend byl zvolen framework Tauri v2 a programovací jazyk Rust. Oproti platformě Electron toto řešení přináší výrazně nižší spotřebu paměti RAM (cca 50 MB oproti stovkám megabajtů), bleskový start a přísný bezpečnostní model oprávnění. V Rustu jsou realizovány nízkoúrovňové systémové integrace, včetně nativní práce se schránkou (podpora wl-paste v Linuxu). Klientské rozhraní je postaveno na knihovně React, jazyce TypeScript a nástroji Vite, což zaručuje typovou bezpečnost a možnost běhu téhož kódu v prohlížeči.

Vícevrstvé vykreslovací jádro (Canvas Engine): Vykreslování je realizováno nad elementem HTML5 Canvas s využitím 2D kontextu. Plátno je rozděleno do tří nezávislých vrstev: statický podklad (mřížka, pozadí stránek A4, importované stránky PDF), prostřední vrstva dokončených vektorových tahů a horní interaktivní vrstva pro právě kreslený tah perem. Přepočet mezi souřadnicemi obrazovky (Screen Space) a virtuálního světa (World Space) zajišťují afinní transformace (škálování a translace). Vyhlazení křivek a teselaci obrysového polygonu tahů proměnlivé tloušťky zajišťuje algoritmus perfect-freehand s adaptací na přítlak stylusu z W3C Pointer Events API. Kompenzaci ostrosti na HiDPI displejích řídí ResizeObserver s dynamickým faktorem DPR.

Správa sešitů a optimalizace zobrazení: V režimu sešitu je plocha organizována do stránek formátu ISO 216 A4 s konstantním poměrem stran $1 : \sqrt{2}$. Pro zachování plynulosti u rozsáhlých sešitů je implementován algoritmus ořezávání mimo záběr (viewport culling), který vykresluje pouze prvky protínající aktuální výřez obrazovky. Pro import studijních materiálů je integrována knihovna PDF.js provádějící asynchronní rasterizaci stránek na pozadí.

Distribuovaná synchronizace a Local-First ukládání: Datový model sešitu je postaven na bezkolidních replikovaných datových typech (CRDT) reprezentovaných knihovnou Yjs. Úpravy se okamžitě ukládají do lokální databáze (IndexedDB / SQLite v režimu WAL). Při síťovém spojení jsou změny přenášeny přes protokol WebSocket na server postavený na frameworku Hono v prostředí Node.js. Díky CRDT dochází k deterministickému sloučení stavu bez editačních kolizí.

Integrace AI asistenta a pedagogický prompting: Uživatel výběrovým nástrojem ohraničí libovolnou oblast plátna. Systém provede její rasterizaci do bezztrátového formátu PNG a odešle dotaz na multimodální Vision-Language API. Systémový prompt navádí model do role sokratovského tutora (vysvětluje postupy krok za krokem a klade doplňující otázky místo pouhého prozrazení výsledku). Ze zadaných poznámek model přes JSON Schema generuje strukturované testovací otázky a flashcards.

Etapizace projektu a testování: Vývoj probíhá v šesti etapách: 1. Návrh architektury a prototyp jádra plátna; 2. Správa vícestránkových sešitů a PDF import; 3. Nativní integrace v Tauri v2 a optimalizace latence; 4. Síťová synchronizace přes WebSockets a CRDT Yjs; 5. Modul AI asistenta a generátor úloh; 6. Profilování výkonu na 60 FPS (Chrome DevTools Profiler), uživatelské testování se studenty a sepsání dokumentace.